8. Develop a Gradient Boosting regression model to predict wine quality based on physicochemical properties. Handle missing values, perform feature scaling, and evaluate the model using MSE, MAE, RMSE, and R-squared.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score
)
```

In [3]:
```
df = pd.read_csv("winequality.csv")
```

In [4]:
```
df.head()
```

Out[4]:

	type	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	densi
0	white	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.00
1	white	6.3	0.30	0.34	1.6	0.049	14.0	132.0	0.994
2	white	8.1	0.28	0.40	6.9	0.050	30.0	97.0	0.995
3	white	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.995
4	white	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.995

In [5]:
```
X = df.drop(columns=["quality", "type"], errors="ignore")
y = df["quality"]
```

In [6]:
```
df.isnull().sum()
```

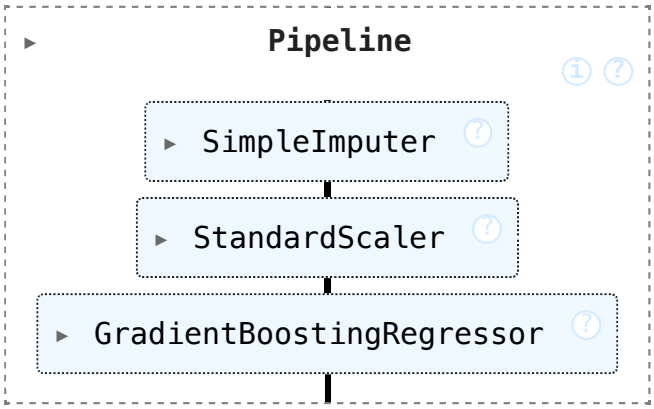
Out[6]:
```
type                    0
fixed acidity          10
volatile acidity        8
citric acid             3
residual sugar          2
chlorides               2
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      9
sulphates               4
alcohol                 0
quality                 0
dtype: int64
```

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

In [8]:
```
model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("regressor", GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=3,
        random_state=42
    ))
])
```

In [9]:
```
model.fit(X_train, y_train)
```

Out[9]:



In [10]:
```
y_pred = model.predict(X_test)
```

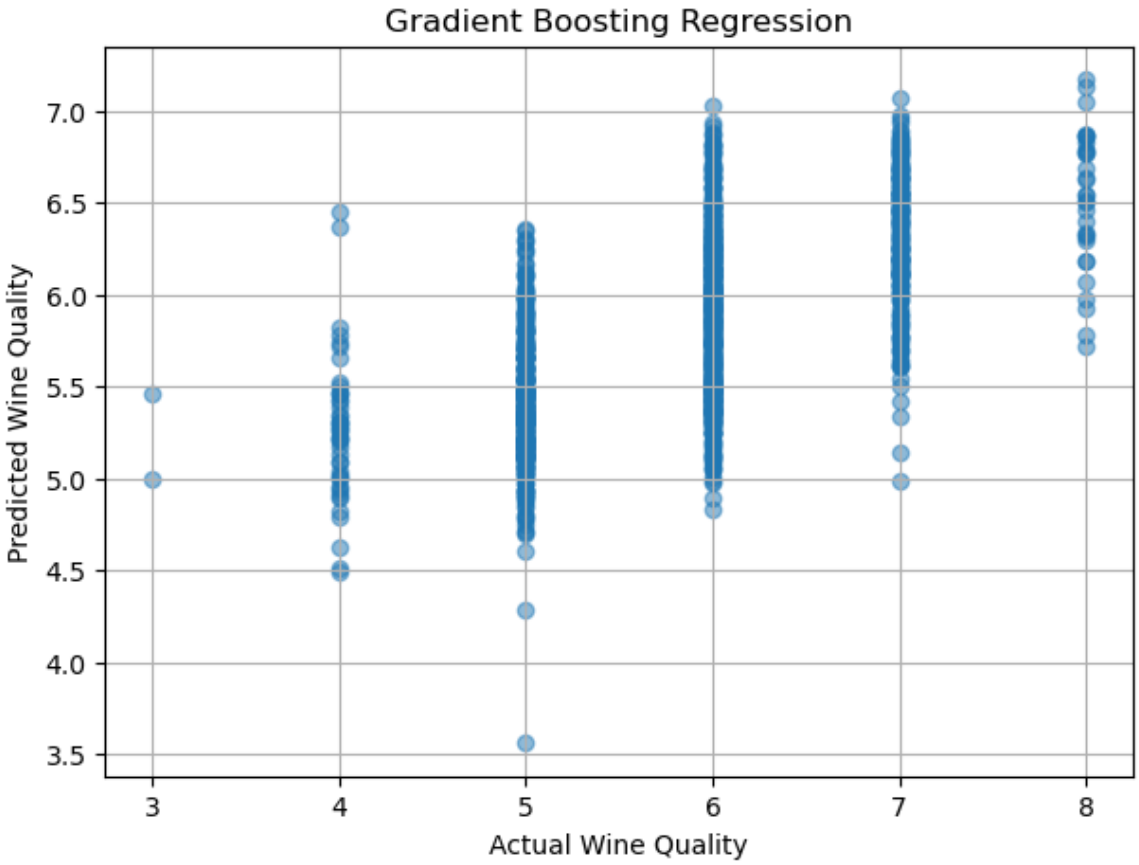
In [11]:
```
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation")
print("MSE :", round(mse, 4))
```

```
print("MAE :", round(mae, 4))
print("RMSE:", round(rmse, 4))
print("R-squared:", round(r2, 4))
```

```
Model Evaluation
MSE : 0.4133
MAE : 0.511
RMSE: 0.6429
R-squared: 0.4219
```

In [12]:
```
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel("Actual Wine Quality")
plt.ylabel("Predicted Wine Quality")
plt.title("Gradient Boosting Regression")
plt.grid(True)
plt.show()
```



In [13]:
```
sample = X_test.iloc[[0]]
print("\nActual quality:", y_test.iloc[0])
print("Predicted quality:", round(model.predict(sample)[0], 2))
```

```
Actual quality: 7
Predicted quality: 6.55
```

9. Build an XGBoost classification model to predict loan defaults based on borrower information and loan characteristics. Preprocess the data by handling missing values, encoding categorical variables, and addressing class imbalance.

In [21]:
```
%pip install pandas numpy matplotlib seaborn scikit-learn xgboost i
```

```
Requirement already satisfied: pandas in ./jupyter_notebook/anaconda
3/lib/python3.13/site-packages (2.3.3)
Requirement already satisfied: numpy in ./jupyter_notebook/anaconda
3/lib/python3.13/site-packages (2.3.5)
Requirement already satisfied: matplotlib in ./jupyter_notebook/anac
onda3/lib/python3.13/site-packages (3.10.6)
Requirement already satisfied: seaborn in ./jupyter_notebook/anacond
a3/lib/python3.13/site-packages (0.13.2)
Requirement already satisfied: scikit-learn in ./jupyter_notebook/an
aconda3/lib/python3.13/site-packages (1.7.2)
Collecting xgboost
  Downloading xgboost-3.4.1-py3-none-macosx_12_0_arm64.whl.metadata
(2.0 kB)
Requirement already satisfied: imbalanced-learn in ./jupyter_noteboo
k/anaconda3/lib/python3.13/site-packages (0.14.0)
Requirement already satisfied: python-dateutil>=2.8.2 in ./jupyter_n
otebook/anaconda3/lib/python3.13/site-packages (from pandas) (2.9.0.
post0)
Requirement already satisfied: pytz>=2020.1 in ./jupyter_notebook/an
aconda3/lib/python3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in ./jupyter_notebook/
anaconda3/lib/python3.13/site-packages (from pandas) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in ./jupyter_noteboo
k/anaconda3/lib/python3.13/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./jupyter_notebook/an
aconda3/lib/python3.13/site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in ./jupyter_notebo
ok/anaconda3/lib/python3.13/site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./jupyter_notebo
ok/anaconda3/lib/python3.13/site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in ./jupyter_noteboo
k/anaconda3/lib/python3.13/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in ./jupyter_notebook/anaco
nda3/lib/python3.13/site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in ./jupyter_noteboo
k/anaconda3/lib/python3.13/site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: scipy>=1.8.0 in ./jupyter_notebook/an
aconda3/lib/python3.13/site-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in ./jupyter_notebook/a
naconda3/lib/python3.13/site-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in ./jupyter_not
ebook/anaconda3/lib/python3.13/site-packages (from scikit-learn) (3.
5.0)
Requirement already satisfied: six>=1.5 in ./jupyter_notebook/anacon
da3/lib/python3.13/site-packages (from python-dateutil>=2.8.2->panda
s) (1.17.0)
Downloading xgboost-3.4.1-py3-none-macosx_12_0_arm64.whl (2.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.4/2.4 MB 1.0 MB/s  0:0
0:02 eta 0:00:01:00:01:01
Installing collected packages: xgboost
Successfully installed xgboost-3.4.1
Note: you may need to restart the kernel to use updated packages.
```

In [22]:
```
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report,
from xgboost import XGBClassifier
```

In [23]:
```
df = pd.read_csv("credit_risk_dataset.csv")

print(df.head())
print(df.isnull().sum())
```

```
   person_age  person_income person_home_ownership  person_emp_lengt
h  \
0          22          59000                  RENT              123.
0
1          21           9600                   OWN                5.
0
2          25           9600              MORTGAGE                1.
0
3          23          65500                  RENT                4.
0
4          24          54400                  RENT                8.
0

  loan_intent loan_grade  loan_amnt  loan_int_rate  loan_status  \
0    PERSONAL          D      35000          16.02            1
1   EDUCATION          B       1000          11.14            0
2     MEDICAL          C       5500          12.87            1
3     MEDICAL          C      35000          15.23            1
4     MEDICAL          C      35000          14.27            1

   loan_percent_income cb_person_default_on_file  cb_person_cred_his
t_length
0                 0.59                         Y
3
1                 0.10                         N
2
2                 0.57                         N
3
3                 0.53                         N
2
4                 0.55                         Y
4
person_age                     0
person_income                  0
person_home_ownership          0
person_emp_length            895
loan_intent                    0
loan_grade                     0
loan_amnt                      0
loan_int_rate               3116
loan_status                    0
loan_percent_income            0
cb_person_default_on_file      0
cb_person_cred_hist_length     0
dtype: int64
```

In [24]:
```
X = df.drop("loan_status", axis=1)
y = df["loan_status"]
```

In [25]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [26]:
```
num_cols = X.select_dtypes(include=["int64", "float64"]).columns
cat_cols = X.select_dtypes(include=["object"]).columns
```

```
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

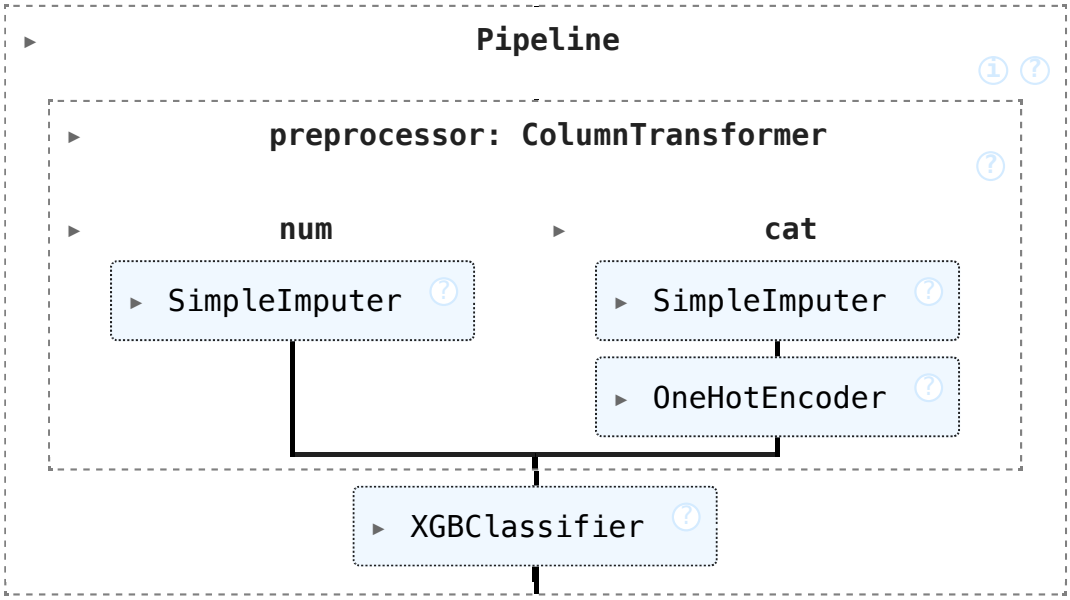
preprocessor = ColumnTransformer([
    ("num", num_pipeline, num_cols),
    ("cat", cat_pipeline, cat_cols)
])
```

In [27]:
```
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
```

In [28]:
```
model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", XGBClassifier(
        n_estimators=200,
        max_depth=4,
        learning_rate=0.1,
        scale_pos_weight=scale_pos_weight,
        eval_metric="logloss",
        random_state=42
    ))
])

model.fit(X_train, y_train)
```

Out[28]:



In [29]:
```
y_pred = model.predict(X_test)
```

In [30]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

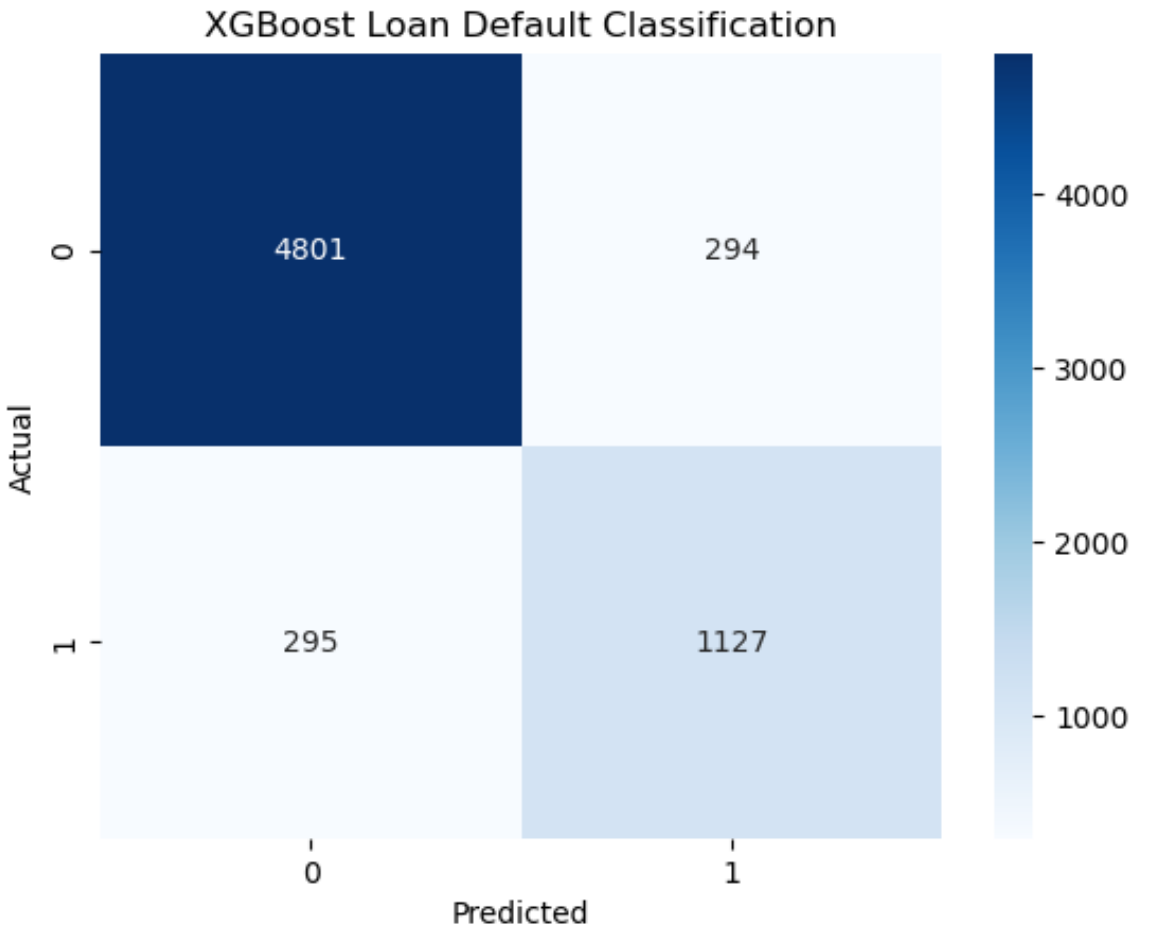
```
Accuracy: 0.9096209912536443
              precision    recall  f1-score   support

           0       0.94      0.94      0.94      5095
           1       0.79      0.79      0.79      1422

    accuracy                           0.91      6517
   macro avg       0.87      0.87      0.87      6517
weighted avg       0.91      0.91      0.91      6517
```

In [31]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("XGBoost Loan Default Classification")
plt.show()
```



10. Solve the Taxi problem using reinforcement learning, where an agent acts as a taxi driver to pick up and drop off passengers efficiently.

In [32]:
```
import numpy as np
import random

n = 4
start = tuple(map(int, input("Enter taxi start position: ").split()
pickup = tuple(map(int, input("Enter passenger position: ").split()
```

```
drop = tuple(map(int, input("Enter destination: ").split()))

# Actions: Up, Down, Left, Right, Pickup, Drop
actions = [(-1,0), (1,0), (0,-1), (0,1)]
Q = np.zeros((n, n, 2, 6))

alpha, gamma, epsilon = 0.1, 0.9, 0.2

# Training
for episode in range(5000):
    pos = start
    passenger = 0

    for step in range(100):
        r, c = pos

        if random.random() < epsilon:
            a = random.randrange(6)
        else:
            a = np.argmax(Q[r, c, passenger])

        nr, nc = r, c
        new_passenger = passenger
        reward = -1
        done = False

        if a < 4:
            nr += actions[a][0]
            nc += actions[a][1]

            if not (0 <= nr < n and 0 <= nc < n):
                nr, nc = r, c
                reward = -10

        elif a == 4:
            if pos == pickup and passenger == 0:
                new_passenger = 1
                reward = 10
            else:
                reward = -10

        elif a == 5:
            if pos == drop and passenger == 1:
                reward = 100
                done = True
            else:
                reward = -10

        future = 0 if done else np.max(Q[nr, nc, new_passenger])

        Q[r, c, passenger, a] += alpha * (
            reward + gamma * future - Q[r, c, passenger, a]
        )

        pos = (nr, nc)
        passenger = new_passenger
```

```
        if done:
            break

# Testing
pos = start
passenger = 0
print("Taxi starts at:", pos)

for step in range(30):
    r, c = pos
    a = np.argmax(Q[r, c, passenger])

    if a < 4:
        nr = r + actions[a][0]
        nc = c + actions[a][1]
        if 0 <= nr < n and 0 <= nc < n:
            pos = (nr, nc)
            print("Taxi moves to:", pos)
        else:
            break

    elif a == 4 and pos == pickup and passenger == 0:
        passenger = 1
        print("Passenger picked up!")

    elif a == 5 and pos == drop and passenger == 1:
        print("Passenger dropped off successfully!")
        break

    else:
        print("Invalid action")
        break
```

```
Taxi starts at: (0, 0)
Taxi moves to: (0, 1)
Taxi moves to: (1, 1)
Taxi moves to: (1, 2)
Passenger picked up!
Taxi moves to: (2, 2)
Taxi moves to: (2, 3)
Taxi moves to: (3, 3)
Passenger dropped off successfully!
```

11. Solve a simple grid world problem using reinforcement earning, where an agent learns to reach a target location while avoiding obstacles.

In [33]:

```
import numpy as np
import random

n = 4
start = tuple(map(int, input("Enter start row and column: ").split(
goal = tuple(map(int, input("Enter goal row and column: ").split())

k = int(input("Enter number of obstacles: "))
```

```
obstacles = []

for i in range(k):
    pos = tuple(map(int, input(f"Obstacle {i+1}: ").split()))
    obstacles.append(pos)

actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
Q = np.zeros((n, n, 4))

alpha, gamma, epsilon = 0.1, 0.9, 0.2

# Training
for episode in range(3000):
    pos = start

    for step in range(100):
        r, c = pos

        if random.random() < epsilon:
            a = random.randrange(4)
        else:
            a = np.argmax(Q[r, c])

        nr = r + actions[a][0]
        nc = c + actions[a][1]

        if not (0 <= nr < n and 0 <= nc < n) or (nr, nc) in obstacl
            nr, nc = r, c
            reward = -10
        elif (nr, nc) == goal:
            reward = 100
        else:
            reward = -1

        Q[r, c, a] += alpha * (
            reward + gamma * (0 if (nr, nc) == goal
                              else np.max(Q[nr, nc])) - Q[r, c, a]
        )

        pos = (nr, nc)

        if pos == goal:
            break

# Testing
pos = start
path = [pos]

for step in range(20):
    if pos == goal:
        break

    r, c = pos
    a = np.argmax(Q[r, c])

    nr = r + actions[a][0]
```

```
    nc = c + actions[a][1]

    if not (0 <= nr < n and 0 <= nc < n) or (nr, nc) in obstacles:
        break

    pos = (nr, nc)
    path.append(pos)

print("Learned Path:", path)
print("Goal Reached:", pos == goal)
```

```
Learned Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2)]
Goal Reached: True
```

In [ ]: